Appendix C: IEEE 830 Template

Software Requirements Specification

for

Tamycs Shield

Version v1

Prepared by YAYA ABDEL SATAR

Lodz University of Technology

MARCH 2026

Table of Contents

Revision History

Name	Date	Reason for Changes	Version
YAYA ABDEL SATAR	4 MARCH	Initial version V1 Initial Requirements Analysis	V1

YAYA ABDEL SATAR	11 MARCH	UseCase and description	V2
YAYA ABDEL SATAR	18 MARCH	CLASS DIAGRAM	V3

1. Problem Statement

Current Situation and Problem:

Academic institutions increasingly rely on digital platforms for conducting examinations. However, standard assessment tools face two critical challenges. First, they are highly vulnerable to network instability; a simple Wi-Fi disconnection can cause a student to lose all their unsaved progress, leading to unfair grading and administrative overhead. Second, they often lack robust anti-cheating mechanisms and data security, making them susceptible to basic exploits (such as SQL injections) or unauthorized access to local test files.

Proposed Solution (Tamycs Shield):

To address these issues, we propose Tamycs Shield, a secure, 3-tier examination system. The system utilizes a modern technology stack to ensure both resilience and security. By implementing automated local caching encrypted with AES-256-GCM, the system guarantees that no student data is lost during network drops. Furthermore, the application will operate in a locked-down environment to prevent unauthorized access to external resources. This project aims to deliver a tamper-proof, highly reliable digital examination environment for universities.

---

2. Overall Description

Product Perspective Tamycs Shield is a self-contained, 3-tier web application designed to replace legacy monolithic assessment tools. It consists of:

Presentation Layer (Client): A dynamic interface built with React for students.

Application Logic Layer (Server): A Node.js backend handling API routing and security logic.

- Data Layer (Database): A PostgreSQL relational database.

User Classes

Administrators (Teachers): Users who create exams, manage question databases, and review student grades.

- Students: End-users who interact exclusively with the secure client interface to take exams.

---

3. Functional Requirements

Note: All requirements are assigned a unique ID for traceability and use the keyword "shall" to indicate an engineering obligation.

- FR-01: The system shall allow Administrators to create an account using a valid university email address and a password.
- FR-02: The Tamycs Server shall sanitize all text inputs from Administrators before storing them in the PostgreSQL database to prevent SQL injection attacks.
- FR-03: Upon the initiation of an exam, the Tamycs Client shall randomly shuffle the order of the questions to generate a unique test sequence for each student.
- FR-04: The Tamycs Client shall automatically save the student's current answers to an encrypted local backup file every 60 seconds.
- FR-05: If the network connection is lost, the Tamycs Client shall attempt to reconnect to the server every 30 seconds without interrupting the student's exam timer.
- FR-06: The system shall calculate and display the final grade to the Administrator within 5 seconds after a student submits their exam.

---

4. Non-Functional Requirements

Note: These requirements define how well the system performs, focusing on security, performance, and reliability.

- NFR-01 (Security): All user and administrator passwords shall be hashed using the bcrypt algorithm before being stored in the database.
- NFR-02 (Performance): The system shall respond to client API requests (such as submitting an answer) in less than 2 seconds.
- NFR-03 (Reliability): The Tamycs Client application shall maintain a crash-free session rate of 99.5% during a standard 2-hour examination period.
- NFR-04 (Security): Data transmission between the React frontend and Node.js backend shall be encrypted using standard HTTPS/TLS protocols.

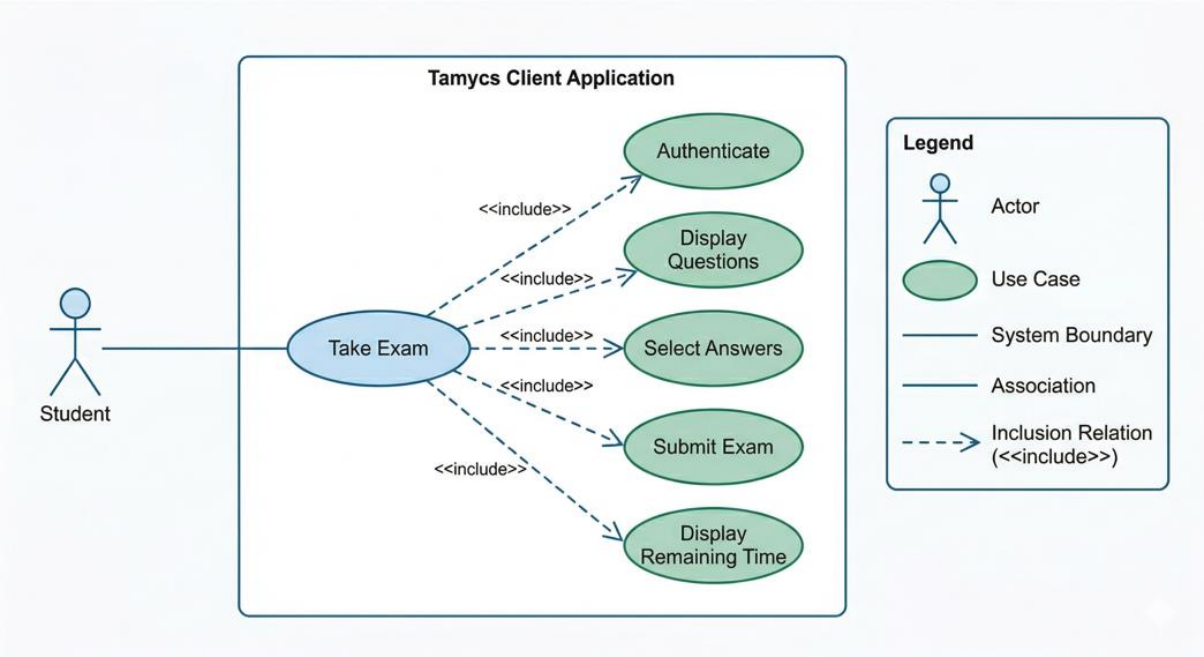
---

5. External Interfaces

5.1 User Interfaces Overview The system features a Graphical User Interface (GUI). The student interface (Tamycs Client) shall operate in a full-screen, distraction-free mode displaying only the current question, multiple-choice answers, and a countdown timer.

5.2 Software Interfaces The React client shall communicate with the Node.js server exclusively via RESTful API endpoints using JSON formatting for all data payloads.

UseCase



6 . System: Tamycs Client Application

This diagram illustrates the functional interactions between the student and the secure examination client interface.

A. Actors Identification

- **Primary Actor: Student** – The end-user who interacts with the React-based client interface to complete an assessment. The student is linked to the core functionality via an **Association** line.

B. Use Case Descriptions

The diagram breaks down the examination process into several specialized modules:

- **Take Exam**: The base use case initiated by the student to start the testing session.

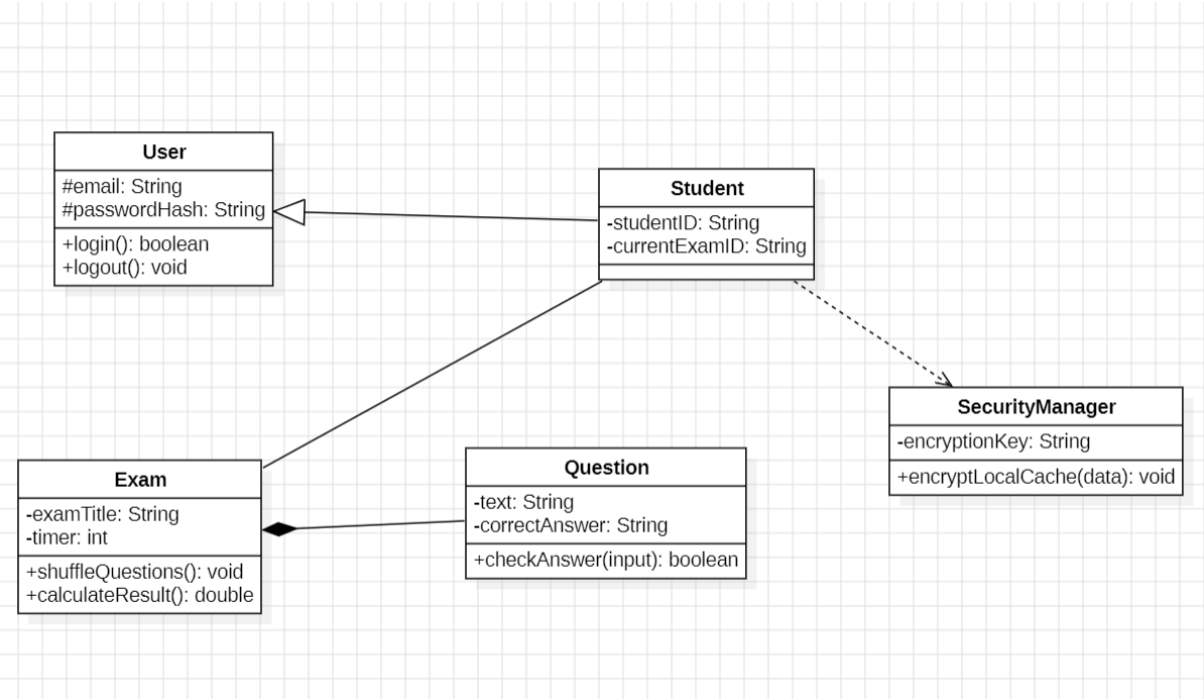
- **Authenticate**: A mandatory security step requiring a valid university email and password before access is granted.
- **Display Questions**: The interface presents the test items, which are randomized to ensure a unique sequence for every student.
- **Select Answers**: The functional requirement allowing the student to choose multiple-choice options.
- **Submit Exam**: The final action where the student transmits their completed test to the Node.js server for grading.
- **Display Remaining Time**: A constant visual countdown timer to assist with student time management.

C. Logical Relations (UML)

The diagram relies exclusively on **Inclusion** relationships to define the system workflow:

- **`<<include>>`**: All dashed arrows point from the **Take Exam** case toward the other specific use cases.
- **Engineering Obligation**: This indicates that the "Take Exam" process cannot be completed without executing all the included sub-steps. In UML terms, these behaviors are required parts of the base use case and are not optional.

CLASS DIAGRAM



Class Structure

1. User (Abstract Class)

Serves as the base for all user types within the system.

- **Attributes:**
  - `# username: String`
  - `# password: String`
- **Method:**
  - `+ login(): boolean`

2. Student

Inherits from the User class and represents the end-user taking the exam.

- **Attributes:**
  - `- studentID: String`
- **Methods:**
  - `+ startExam()`
  - `+ submitExam()`

3. Exam

Represents the assessment entity.

- **Attributes:**

    - `- examID: String`
    - `- duration: int`
- **Method:**
    - `+ validateSubmission()`

4. Question

Details the individual items within an exam.

- **Attributes:**
    - `- questionID: String`
    - `- content: String`
- **Method:**
    - `+ checkAnswer()`

5. SecurityManager

Handles critical security constraints, such as AES-256 encryption for local caching.

- **Methods:**
    - `+ encryptData()`
    - `+ monitorActivity()`

Key Relationships

The diagram utilizes precise UML notations to define interactions:

- **Inheritance (Generalization)**: The **Student** is a specialization of the **User** (indicated by a hollow arrow pointing toward User).
- **Composition**: An **Exam** is composed of multiple **Questions**. If the exam is deleted, the associated questions are also destroyed (indicated by a solid diamond on the Exam side).
- **Association**: The **Student** takes an **Exam** (represented by a simple line connecting the two classes).
- **Dependency**: The **Student** relies on the **SecurityManager** for data protection (represented by a dashed line with an arrow).

Visibility Standards

- + **Public**: Accessible by any other class.
- – **Private**: Accessible only within the class (Encapsulation).
- # **Protected**: Accessible by the class and its subclasses (used for Inheritance between User and Student).